

TÀI LIỆU THIẾT KẾ KIẾN TRÚC HỆ THỐNG

(Software Architecture Document - SAD)

Dự án:	Hệ thống Quản lý Kho Thông minh (Warehouse Management System)
Kiến trúc:	Clean Architecture (Go)
Tác giả:	Hà Tấn Minh
Ngày lập:	12 tháng 06 năm 2026
Phiên bản:	v1.0.0

1. Tổng quan hệ thống

Tài liệu Thiết kế Kiến trúc Phần mềm (SAD) này mô tả toàn diện về mặt kỹ thuật, mô hình dữ liệu và cách tổ chức mã nguồn của Hệ thống Quản lý Kho Thông minh. Hệ thống được xây dựng trên ngôn ngữ lập trình Go (Golang), áp dụng nghiêm ngặt nguyên lý **Clean Architecture** để đảm bảo khả năng bảo trì, độc lập với cơ sở dữ liệu và framework, đồng thời dễ dàng mở rộng quy mô.

Mục tiêu cốt lõi của mã nguồn backend là phục vụ đồng thời cho **Web Client** (Quản lý, Báo cáo, Giám sát) và **Android App** (Quét mã QR/Rfid thực địa của tủ kho), đảm bảo tính toàn vẹn dữ liệu và cập nhật trạng thái tồn kho theo thời gian thực (Real-time sync).

2. Sơ đồ kiến trúc tổng thể (Architecture Diagram)

Hệ thống được thiết kế theo mô hình đa tầng (Multi-tier Enterprise Architecture), tách biệt rõ ràng giữa thành phần giao diện, tầng điều phối trung gian và hạ tầng lưu trữ. Sự tương tác giữa các thành phần được thể hiện chi tiết qua luồng dữ liệu dưới đây:



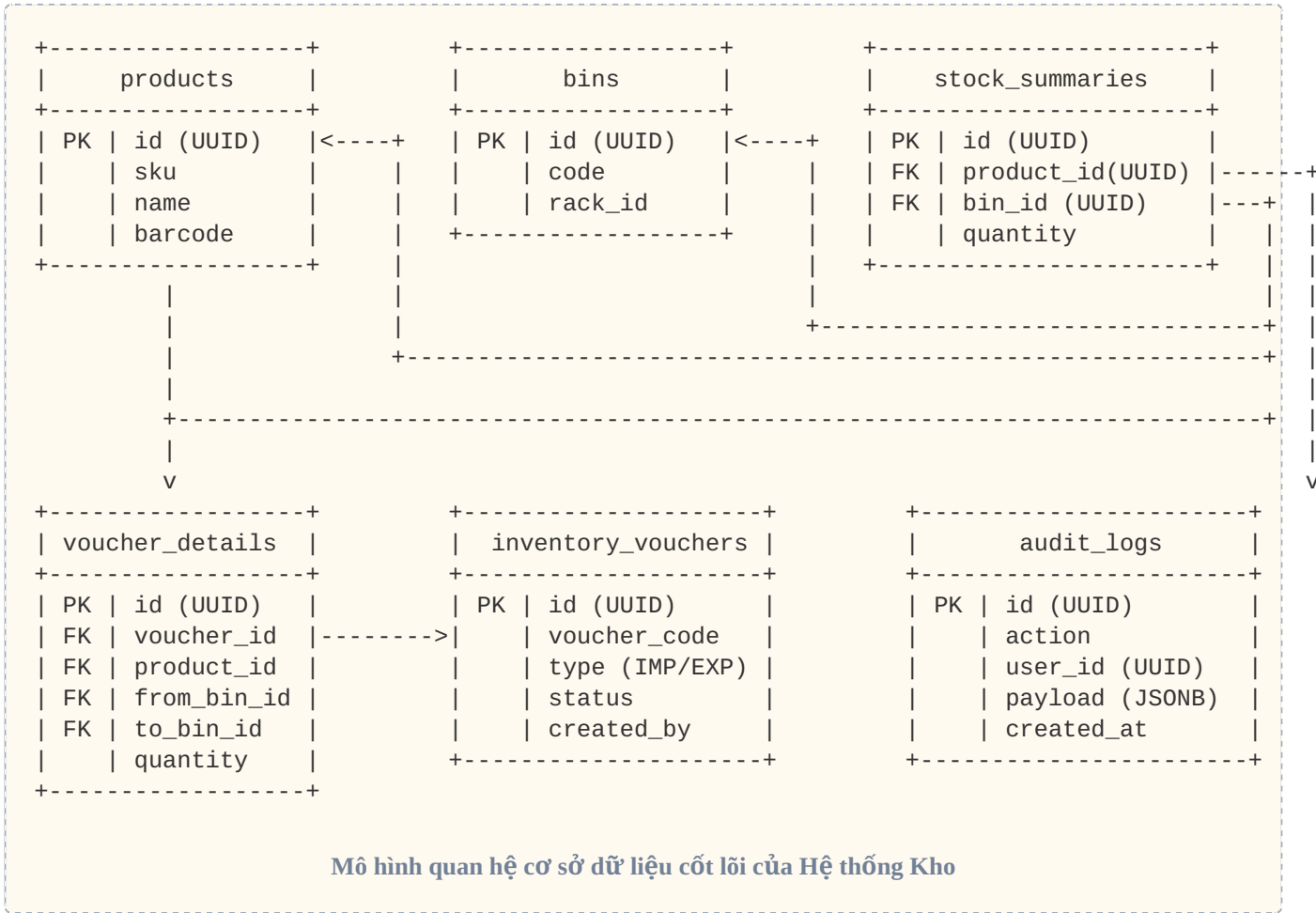
Mô tả sự tương tác giữa các thành phần hạ tầng:

- **Giao tiếp HTTP RESTful API:** Cả Web Client và Android App đều thực hiện các yêu cầu đồng bộ (như Đăng nhập, Tạo sản phẩm, Xem danh sách vị trí kho) thông qua giao thức HTTP REST API sử dụng định dạng JSON. Tất cả các yêu cầu qua cửa ngõ này đều được bảo vệ bởi Middleware xác thực JSON Web Token (JWT) và phân quyền 3 vai trò nghiêm ngặt (Admin, Quản lý kho, Thủ kho).
- **Kết nối thời gian thực WebSocket:** Khi Android App thực hiện quét mã QR/RFID để nhập/xuất kho hoặc cập nhật phiếu, một sự kiện được bắn lên Backend. Server xử lý nghiệp vụ và đẩy thông điệp cập nhật xuống WebSocket Hub. Hub thực hiện broadcast trạng thái tồn kho mới tới toàn bộ các kết nối đang hoạt động giúp màn hình Web Client của Quản lý cập nhật tức thì mà không cần tải lại trang.
- **Lớp bộ nhớ đệm Redis:** Đóng vai trò kép trong hệ thống. Thứ nhất, Redis lưu trữ cache các thông tin tĩnh hoặc dữ liệu truy vấn tần suất cao như danh mục sản phẩm, cấu hình phân cấp kho giúp tăng tốc phản hồi **O(1)**. Thứ hai, Redis tận dụng cơ chế Pub/Sub để điều phối và đồng bộ hóa các sự kiện thời gian thực giữa các instance backend trong môi trường chịu tải cao.
- **Cơ sở dữ liệu PostgreSQL:** Là trung tâm lưu trữ dữ liệu vĩnh viễn (Data Persistence Layer). PostgreSQL lưu giữ các thực thể có tính toàn vẹn cao, sử dụng các ràng buộc khóa ngoại chặt chẽ và lưu lịch sử biến động bằng trường JSONB tối ưu.

3. Thiết kế thực thể mối quan hệ dữ liệu (Database Schema Design - ERD)

Hệ thống dữ liệu lưu trữ trong PostgreSQL được chuẩn hóa để loại bỏ dư thừa, tối ưu hóa hiệu năng truy vấn cho các cấu trúc phân cấp phức tạp (Warehouse → Zone → Rack → Bin) và quản lý chặt chẽ dòng dịch chuyển vật tư.

3.1. Sơ đồ thực thể mối quan hệ bằng mã ký họa (Text-based ERD)



3.2. Đặc tả chi tiết các bảng hệ thống

Bảng 1: products (Thông tin sản phẩm)

Lưu trữ danh mục định danh vật tư sản phẩm trong hệ thống kho.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghĩa nghĩa
id	UUID	PRIMARY KEY	Khóa chính tự động sinh định danh sản phẩm.
sku	VARCHAR(50)	UNIQUE, NOT NULL	Mã định danh quản lý lưu kho duy nhất (Stock Keeping Unit).
name	VARCHAR(255)	NOT NULL	Tên hiển thị chi tiết của sản phẩm.
qr_code	VARCHAR(255)	UNIQUE, NULLABLE	Dữ liệu chuỗi mã hóa cho tem mã QR của sản phẩm.
rfid_uid	VARCHAR(100)	UNIQUE, NULLABLE	Mã định danh phần cứng chip RFID gắn trên sản phẩm.
min_stock	INT	DEFAULT 0	Ngưỡng tồn kho tối thiểu để kích hoạt hệ thống cảnh báo.

Bảng 2: bins (Ô lưu trữ nhỏ nhất trong kho)

Đại diện cho các ô/vị trí cụ thể nằm trên các dãy kệ (Racks), thuộc phân khu (Zones) của Nhà kho (Warehouses).

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghĩa nghĩa
id	UUID	PRIMARY KEY	Định danh duy nhất của ô kho.
code	VARCHAR(50)	UNIQUE, NOT NULL	Mã định vị ô (Ví dụ: W1-Z2-R3-B05).
rack_id	UUID	FOREIGN KEY	Khóa ngoại liên kết tới bảng kệ chứa (racks.id).
max_capacity	INT	NOT NULL	Sức chứa số lượng sản phẩm tối đa của ô kho.

Bảng 3: stock_summaries (Bảng tổng hợp tồn kho thực tế - Real-time Stock)

Lưu trạng thái số lượng tồn thực tế của từng loại sản phẩm tại từng ô vị trí cụ thể.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghĩa nghĩa
id	UUID	PRIMARY KEY	Khóa chính thực thể.
product_id	UUID	FOREIGN KEY	Khóa ngoại liên kết bảng products(id).
bin_id	UUID	FOREIGN KEY	Khóa ngoại liên kết bảng bins(id).
quantity	INT	NOT NULL	Số lượng thực tế đang nằm tại vị trí này (Yêu cầu ≥ 0).
updated_at	TIMESTAMP	NOT NULL	Thời điểm cập nhật số lượng tồn gần nhất.

Bảng 4: inventory_vouchers (Chứng từ/Phiếu xuất nhập kho)

Quản lý các thông tin hành chính của một đợt luân chuyển hàng hóa.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghĩa nghĩa
id	UUID	PRIMARY KEY	Khóa chính của phiếu kho.
voucher_code	VARCHAR(50)	UNIQUE, NOT NULL	Mã số phiếu định danh sinh tự động (Ví dụ: PX-2026-0001).
type	VARCHAR(20)	NOT NULL	Loại phiếu: IMPORT (Nhập), EXPORT (Xuất), TRANSFER (Điều chuyển).
status	VARCHAR(30)	NOT NULL	Trạng thái xử lý: PENDING, PROCESSING, COMPLETED, CANCELLED.
created_by	UUID	FOREIGN KEY	Mã người tạo phiếu (Khóa ngoại liên kết bảng users).
created_at	TIMESTAMP	NOT NULL	Thời gian lập phiếu ban đầu.

Bảng 5: voucher_details (Chi tiết mặt hàng trong phiếu)

Lưu danh sách chi tiết các mặt hàng cùng vị trí đi và đến của hàng hóa trong phiếu.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghĩa nghĩa
id	UUID	PRIMARY KEY	Định danh dòng chi tiết.
voucher_id	UUID	FOREIGN KEY	Khóa ngoại liên kết chặt chẽ tới inventory_vouchers(id).
product_id	UUID	FOREIGN KEY	Khóa ngoại liên kết vật tư products(id).
from_bin_id	UUID	FOREIGN KEY, NULL	Ô xuất phát (Null nếu là phiếu Nhập kho mới).
to_bin_id	UUID	FOREIGN KEY, NULL	Ô đích đến (Null nếu là phiếu Xuất kho hẳn).
quantity	INT	NOT NULL	Số lượng yêu cầu thực hiện luân chuyển.

Bảng 6: audit_logs (Nhật ký kiểm toán hệ thống nghiệp vụ nâng cao)

Lưu lại toàn bộ lịch sử thao tác của người dùng nhằm phục vụ mục đích kiểm toán, truy vết lỗi hoặc gian lận.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả nghĩa nghĩa
id	UUID	PRIMARY KEY	Định danh log duy nhất.
action	VARCHAR(100)	NOT NULL	Hành động hệ thống (Ví dụ: PRODUCT_CREATE, STOCK_OVERFLOW, VOUCHER_APPROVE).
user_id	UUID	NOT NULL	ID của người thực hiện hành động truy vết.
payload	JSONB	NOT NULL	Dữ liệu động lưu cấu trúc phức tạp dạng JSON nhị phân: chứa toàn bộ snapshot dữ liệu cũ, dữ liệu mới thay đổi, thông tin thiết bị IP, User-Agent, hoặc mã lỗi đi kèm.
created_at	TIMESTAMP	DEFAULT NOW()	Thời điểm ghi nhận thao tác chính xác.

4. Cấu trúc thư mục mã nguồn Clean Architecture (Go)

Mã nguồn backend Go được tổ chức theo triết lý **Clean Architecture** của Robert C. Martin. Cấu trúc này chia mã nguồn thành các vòng tròn đồng tâm, hướng phụ thuộc luôn đi từ ngoài vào trong: các tầng hạ tầng bên ngoài chỉ được phụ thuộc vào tầng logic cốt lõi bên trong, tầng domain tuyệt đối không biết gì về framework, database hay giao diện bên ngoài.

4.1. Sơ đồ cây thư mục dự án

```
warehouse-backend/  
├── cmd/  
│   └── server/  
│       └── main.go           # Điểm khởi tạo ứng dụng, đọc cấu hình và kích hoạt  
DI Container  
├── config/  
│   └── config.go           # Đọc tham số cấu hình từ môi trường hệ thống (.env)  
├── db/  
│   └── migrations/  
│       └── 001_init.sql     # Kịch bản SQL cấu trúc bảng dữ liệu gốc ban đầu  
├── internal/  
│   ├── domain/            # TẦNG DOMAIN: Trung tâm cốt lõi (Entities,  
Interfaces)  
│   │   └── product.go      # Khai báo Struct Sản phẩm & Quy tắc Validate nghiệp  
vụ tính
```



```

|   |   |   | bin.go           # Cấu trúc thực thể ô chứa kho
|   |   |   | voucher.go      # Định nghĩa thực thể chứng từ phiếu xuất nhập
|   |   |   | audit.go        # Cấu trúc Audit Log lưu trữ
|   |   |   |
|   |   |   | repository/      # TẦNG REPOSITORY: Thực thi tương tác lưu trữ
(Database Access)
|   |   |   | postgres/
|   |   |   | | product_repo.go # Viết mã SQL / GORM thao tác CRUD bảng products
|   |   |   | | voucher_repo.go # Thao tác nghiệp vụ dữ liệu phiếu kho dữ liệu
|   |   |   | | redis/
|   |   |   | | cache_repo.go  # Thực thi đọc/ghi dữ liệu tạm thời, Pub-Sub sự kiện
|   |   |   |
|   |   |   | usecase/         # TẦNG USECASE / SERVICE: Logic nghiệp vụ ứng dụng
điều phối
|   |   |   | | product_ucase.go # Xử lý Logic hàng hóa, kiểm tra trùng lặp SKU
|   |   |   | | voucher_ucase.go # Nghiệp vụ duyệt phiếu, trừ/cộng dồn tồn kho, bán
cảnh báo
|   |   |   |
|   |   |   | delivery/        # TẦNG DELIVERY / HANDLER: Giao tiếp môi trường bên
ngoài (I/O)
|   |   |   | | http/
|   |   |   | | | auth_handler.go # Tiếp nhận Login REST, giải mã và trả JWT Token
|   |   |   | | | product_handler.go # Tiếp nhận yêu cầu REST của Sản phẩm, kiểm tra định
dạng nhập
|   |   |   | | | middleware.go  # Bộ lọc xác thực JWT, chặn quyền RBAC trước khi vào
handler
|   |   |   | | | websocket/
|   |   |   | | | | hub.go       # Quản lý kết nối Client Pool hoạt động thời gian
thực
|   |   |   | | | | stock_socket.go # Tiếp nhận nâng cấp kết nối và truyền tải sự kiện
real-time
|   |   |   | | go.mod          # Quản lý thư viện module phụ thuộc của Golang

```

4.2. Giải thích chi tiết chức năng và trách nhiệm từng tầng

1. Tầng Domain (internal/domain)

Đây là tầng nằm ở lõi trung tâm của hệ thống. Nó chứa các định nghĩa thực thể dữ liệu gốc (Structs) và các Interface đại diện cho các hành vi lưu trữ hoặc dịch vụ. Tầng này độc lập hoàn toàn, không import bất cứ thư viện bên thứ ba nào liên quan đến Web framework (như Gin) hay ORM (như GORM). Điều này đảm bảo quy tắc nghiệp vụ cốt lõi không bao giờ thay đổi dù hệ thống có quyết định đổi từ PostgreSQL sang MongoDB hay MySQL trong tương lai.

2. Tầng Repository (internal/repository)

Tầng này chịu trách nhiệm trực tiếp về mặt dữ liệu persistence (Data Access Object). Các struct trong tầng này sẽ triển khai (implement) các Interface được định nghĩa trong tầng Domain. Tại đây, lập trình viên viết các câu lệnh SQL thuần hoặc sử dụng GORM để truy vấn cơ sở dữ liệu PostgreSQL, hoặc gọi thư viện go-redis để thực hiện việc cache và Pub/Sub dữ liệu. Mọi lỗi phát sinh từ driver cơ sở dữ liệu sẽ được tầng này bắt lại và chuyển hóa thành các dạng lỗi nghiệp vụ chuẩn hóa trước khi đẩy lên tầng trên.

3. Tầng Usecase / Service (internal/usecase)

Nơi tập trung toàn bộ "bộ não" điều phối của ứng dụng. Tầng Usecase nhận dữ liệu đã qua kiểm tra thô từ tầng Delivery, áp dụng các quy tắc nghiệp vụ logic thực tế của doanh nghiệp. Ví dụ: Khi xử lý hàm ApproveVoucher (Duyệt phiếu xuất kho), tầng Usecase sẽ gọi ProductRepository kiểm tra xem sản phẩm có tồn tại không, gọi StockRepository kiểm tra số lượng tồn hiện tại của ô kho xuất phát xem có đủ điều kiện xuất không ($Qty_{\text{Total}} \geq Qty_{\text{Used}}$). Nếu thỏa mãn, nó thực hiện trừ tồn kho trong một Database Transaction, ghi một dòng vào AuditLog với payload dạng JSONB, và cuối cùng kích hoạt sự kiện thông báo qua WebSocket thông qua Repository để các client đồng bộ.

4. Tầng Delivery / Handler (internal/delivery)

Là tầng tiếp xúc trực tiếp với thế giới bên ngoài, chịu trách nhiệm tiếp nhận và phản hồi dữ liệu (Input/Output). Hệ thống hỗ trợ song song hai giao thức kết nối tại đây: giao thức HTTP RESTful (sử dụng Gin-Gonic làm động cơ định tuyến) phục vụ các thao tác đồng bộ thông thường, và kết nối WebSocket bền vững phục vụ truyền tải sự kiện real-time. Nhiệm vụ duy nhất của tầng này là parse dữ liệu từ Request (JSON body, query params), thực thi các Middleware xác thực JWT/phân quyền, chuyển giao dữ liệu sạch cho tầng Usecase xử lý, và đóng gói kết quả trả về bằng HTTP Status Code (200 OK, 210 Created, 400 Bad Request, v.v.) chuẩn RESTful hoặc khung truyền tin WebSocket.

5. Đánh giá khả năng bảo trì và mở rộng hệ thống (Maintainability & Scalability)

Việc phân tách mã nguồn Go thành 4 tầng rõ rệt trong cấu trúc Clean Architecture mang lại những lợi ích kỹ thuật vượt trội, minh chứng cho một hệ thống chuẩn doanh nghiệp bền vững:

- **Dễ dàng viết Unit Test độc lập:** Do tất cả sự giao tiếp giữa tầng Usecase và hạ tầng (Repository, DB) đều thông qua Interface của Domain, lập trình viên có thể dễ dàng sinh mã Mocking (sử dụng mockgen) để kiểm

thử toàn bộ logic nghiệp vụ của tầng Usecase một cách độc lập mà không cần phải kết nối trực tiếp vào cơ sở dữ liệu thật, giúp tốc độ chạy CI/CD tăng lên đáng kể.

- **Phát triển song song (Parallel Development):** Đội ngũ kỹ sư có thể làm việc độc lập mà không bị nghẽn (block). Lập trình viên Android và Frontend chỉ cần nhìn vào danh sách đặc tả API của tầng Delivery để phát triển giao diện; lập trình viên Core Backend tập trung tối ưu thuật toán trong tầng Usecase; trong khi kỹ sư hệ thống cơ sở dữ liệu có thể tinh chỉnh các câu lệnh SQL Index trong tầng Repository.
- **Khả năng mở rộng chiều ngang (Horizontal Scaling):** Việc đưa lớp bộ nhớ đệm Redis và cơ chế Pub/Sub vào kiến trúc giúp backend hoàn toàn không trạng thái (Stateless). Khi số lượng thiết bị Android App quét mã thực địa đồng thời tăng lên đột biến vào các mùa cao điểm, hệ thống có thể dễ dàng mở rộng bằng cách thêm mới các Node backend đứng sau một bộ cân bằng tải (Load Balancer) mà không sợ mất đồng bộ trạng thái kết nối thời gian thực WebSocket.